

AI 短视频自动化生产流水线

基于 FastAPI + Vue 3 + pyjianYingDraft 的技术实现方案

1. 方案概述

本方案旨在构建一套高度解耦、可编程的短视频生产系统。系统通过 AI 驱动创意生成，结合人工素材制作，最终由代码自动合成剪映草稿工程。该架构通过标准化 JSON 协议连接各个模块，确保了生产的高效性与后续的可扩展性。

核心阶段定义

- 模块一：剧本中枢 (LLM Engine)** - 将灵感转化为结构化 JSON 施工图。
- 模块二：素材调度 (Asset Logic)** - 自动关联本地物理文件，进行路径映射与状态校验。
- 模块三：组装流水 (Draft Engine)** - 解析 JSON 协议并驱动 pyjianYingDraft 合成工程。

2. 技术栈选型

维度	技术选型	关键原因
后端核心	FastAPI (Python 3.10+)	异步性能高，自动生成 API 文档，与 pyjianYingDraft 零成本集成。
前端框架	Vue 3 (Vite + Pinia)	Composition API 逻辑复用性强，适合管理复杂的剧本编辑状态。

渲染引擎	pyJianYingDraft	纯 Python 操作剪映草稿协议，无需 UI 模拟，秒级合成。
通信协议	RESTful API (JSON)	标准化数据交互，支持前后端分离部署。
存储/计算	PVE + fnOS + NFS	计算在容器，存储在 NAS，确保大容量素材的读写效率。

3. 核心数据协议 (Script JSON V1)

所有模块均围绕 `script.json` 进行读写。该协议是系统的唯一事实来源。

```
{
  "project_id": "UUID",
  "config": { "title": "五月星象", "resolution": "9:16" },
  "scenes": [
    {
      "id": 1,
      "narration": "文案内容...",
      "visual_prompt": "SD/MJ 生图提示词",
      "assets": {
        "video_path": "", // 初始为空，模块二回填
        "audio_path": "", // 初始为空，模块二回填
        "duration": 0 // 模块二根据音频自动计算
      },
      "edit": { "transition": "fade", "animation": "zoom_in" }
    }
  ]
}
```

4. 后端架构 (FastAPI)

4.1 核心 API 设计

Endpoint	Method	功能描述
/api/script/generate	POST	接收灵感，调用 LLM 返回初始结构化剧本 JSON。
/api/asset/sync	POST	扫描指定项目文件夹，重命名素材并回填路径至 JSON。
/api/draft/build	POST	解析填充后的 JSON，通过 pyJianYingDraft 保存为剪映工程。

4.2 业务逻辑封装

后端采用 **Service Pattern**，将 LLM 交互、文件处理和草稿生成解耦。利用 Python 的 `pydantic` 进行严格的类型校验，确保 `script.json` 在流转过程中的健壮性。

5. 前端架构 (Vue 3)

5.1 响应式状态管理

使用 **Pinia** 存储当前项目的剧本数据。当用户在 UI 上微调文案或更换转场效果时，前端会实时更新 JSON 状态，并同步至后端。

5.2 核心组件模块

- **InspirationInput:** 支持 Markdown 的灵感输入框，集成了提示词模板。
- **SceneTimeline:** 可视化的镜头流，支持拖拽排序和实时素材状态检查（绿灯表示素材已就绪）。
- **ConfigPanel:** 调整视频比例、全局背景音乐音量等参数。

6. 落地执行计划

第一步 (环境部署): 在 PVE 开辟 LXC 容器，挂载 fnOS 素材盘。安装 Python 环境与 Node.js 环境。

第二步 (后端打通): 编写 FastAPI 骨架，集成 pyJianYingDraft。实现“给定路径 -> 生成空草稿”的基础功能。

第三步 (前端联动): 搭建 Vue 3 界面，实现与后端 API 的数据交互，完成“灵感 -> JSON 展示”闭环。

第四步 (自动化闭环): 编写文件夹 Watcher 逻辑，实现素材放入文件夹后前端 UI 自动变绿的即时反馈。